

Tux: Efficient Drop-in Networking for Database Systems

Xinjing Zhou Viktor Leis Xiangyao Yu Michael Stonebraker

MIT CSAIL

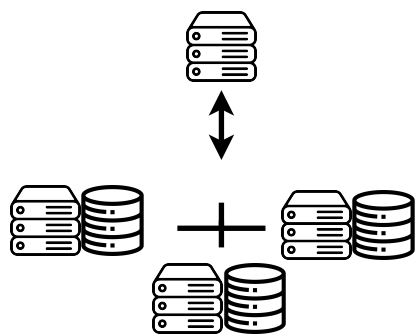
TUM

UW-Madison

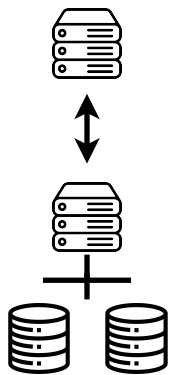
MIT CSAIL



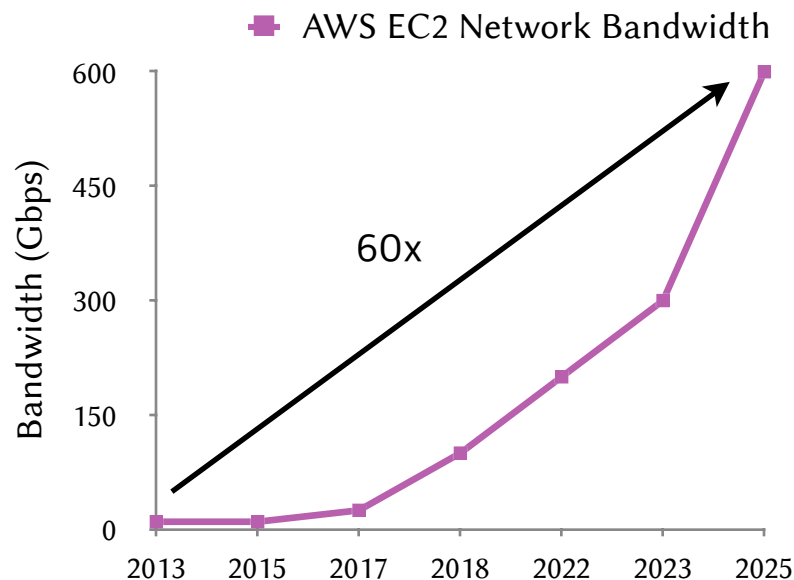
Networking is now central in DBMSs



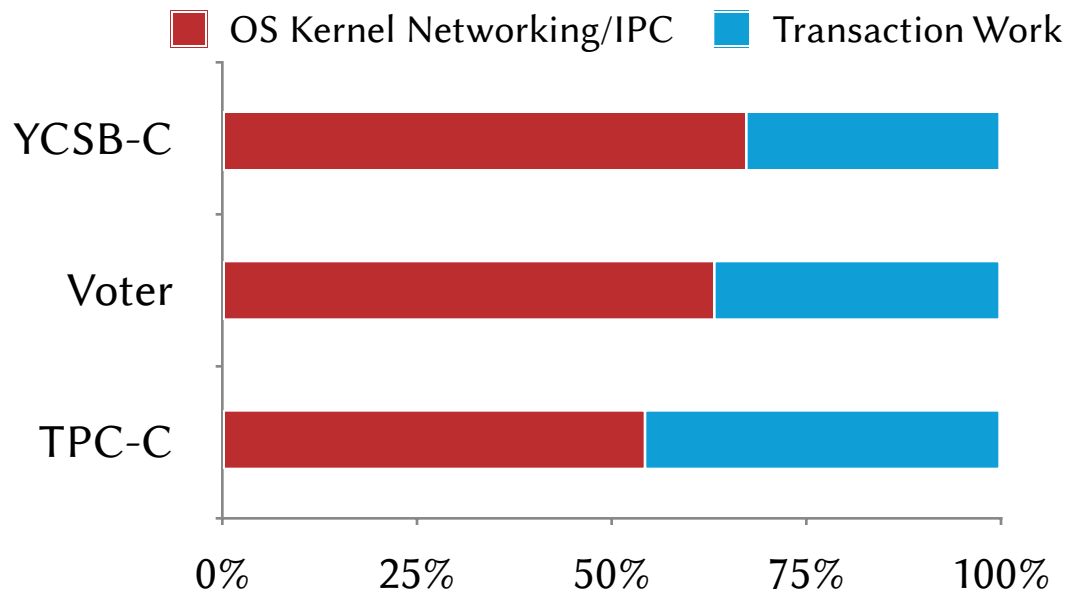
Replication
Distributed Transactions



Disaggregation



OS networking dominates OLTP CPU time

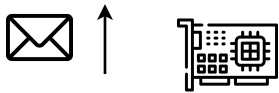


CPU Cycle Breakdown of a
Memory-optimized OLTP DBMS [1]

Kernel stack vs kernel bypass

Userspace

Kernel



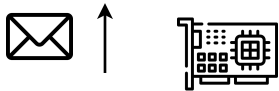
Linux Stack

Kernel stack vs kernel bypass

Userspace

Kernel

Kernel NIC Driver



Linux Stack

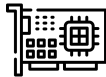
Kernel stack vs kernel bypass

Userspace

Kernel

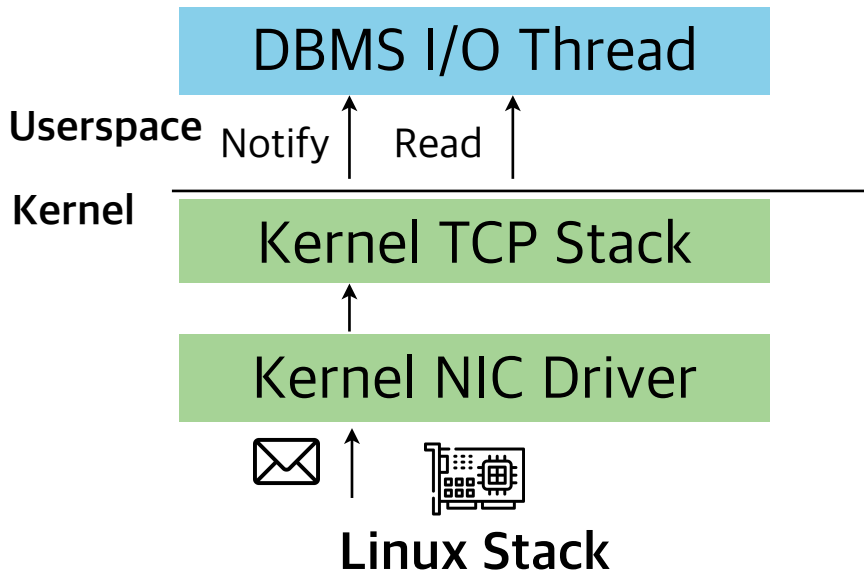
Kernel TCP Stack

Kernel NIC Driver

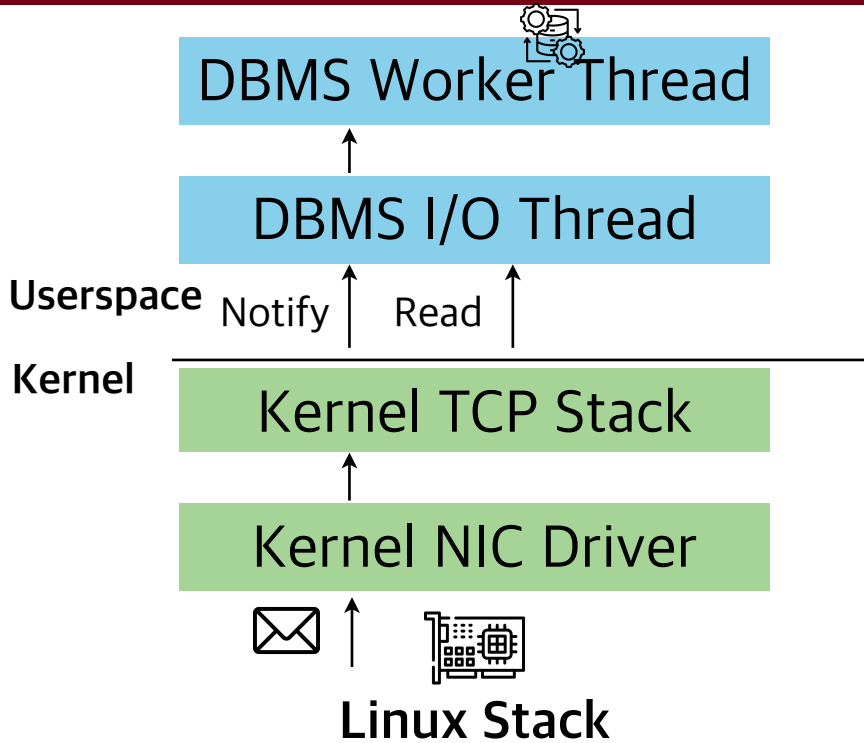


Linux Stack

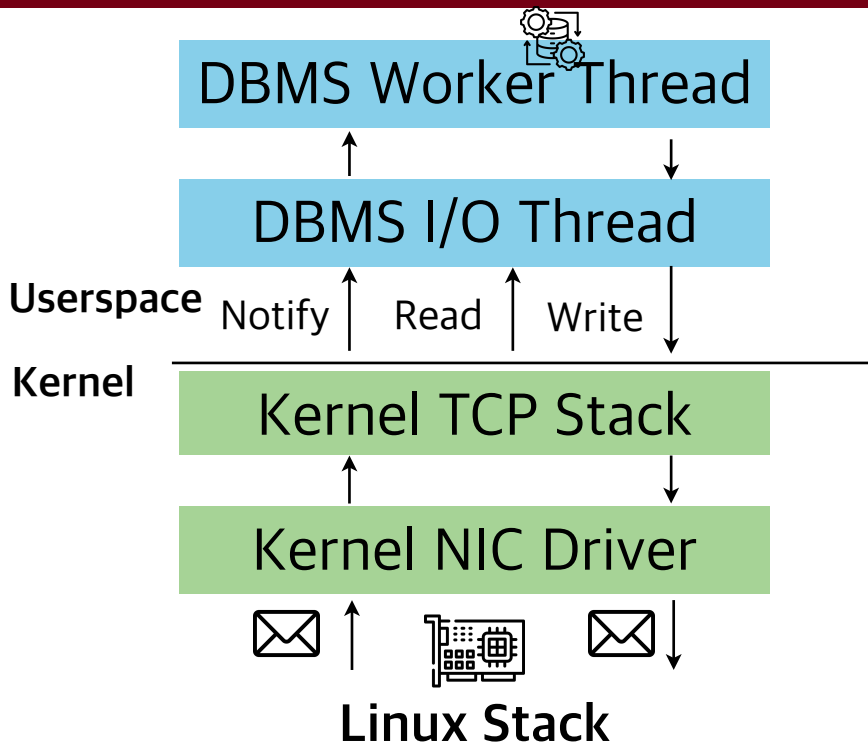
Kernel stack vs kernel bypass



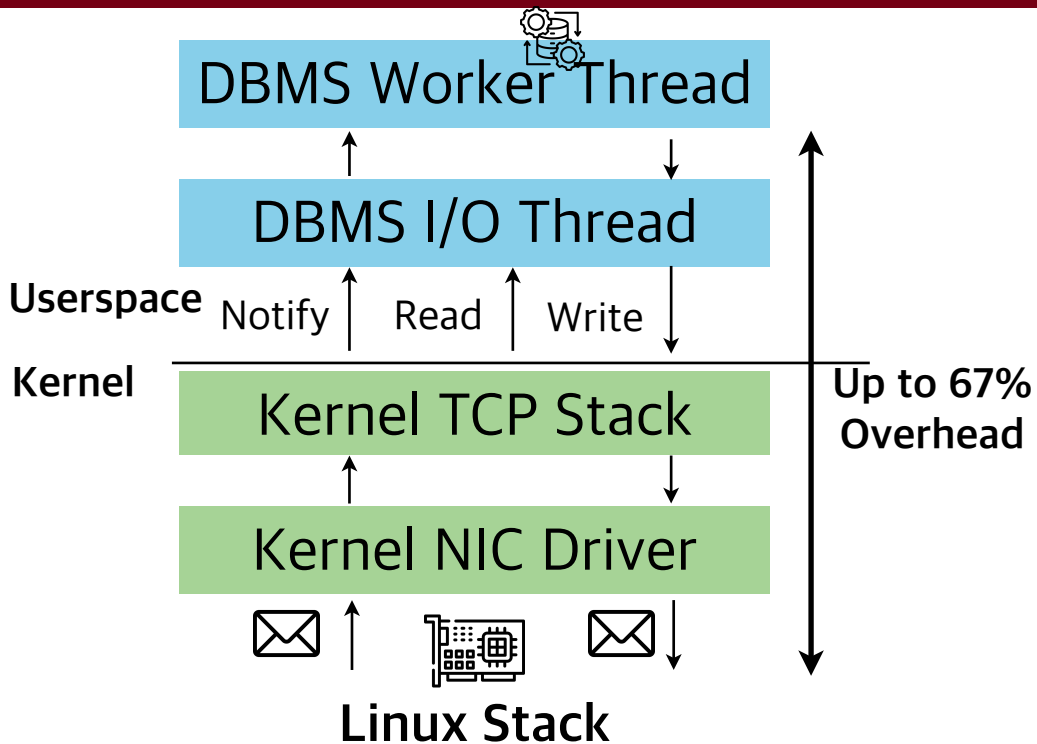
Kernel stack vs kernel bypass



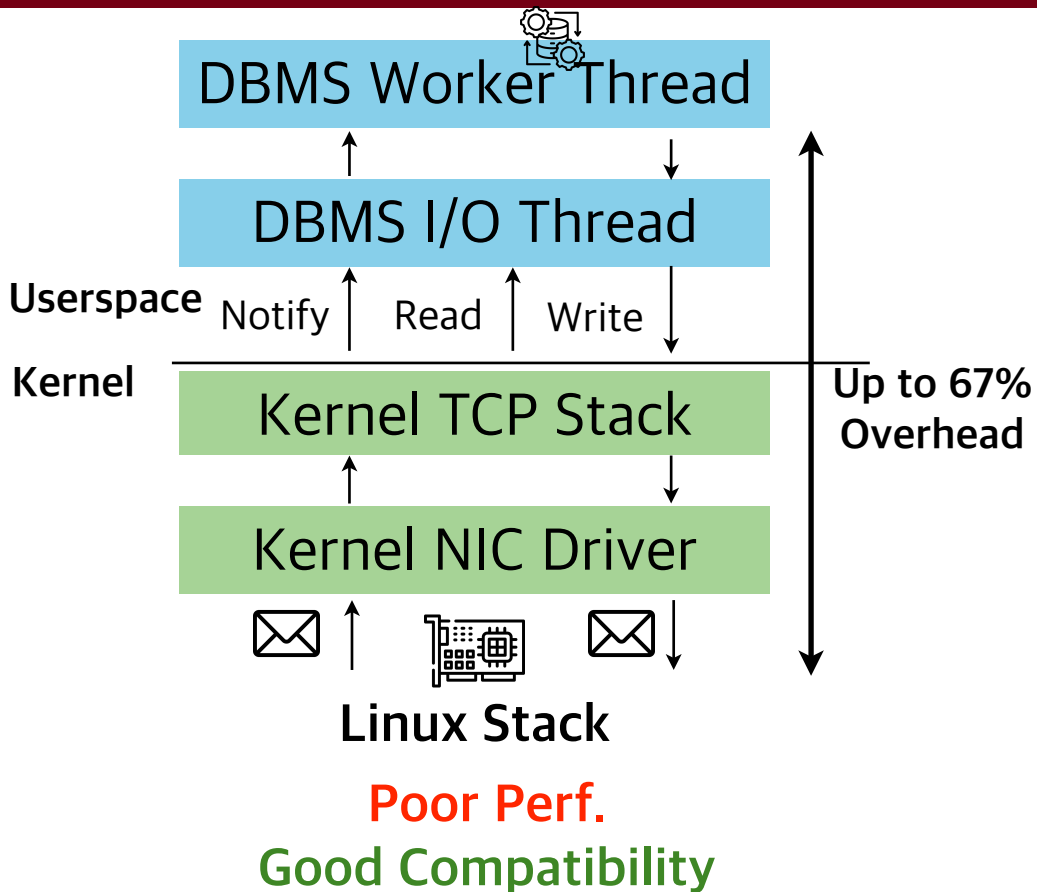
Kernel stack vs kernel bypass



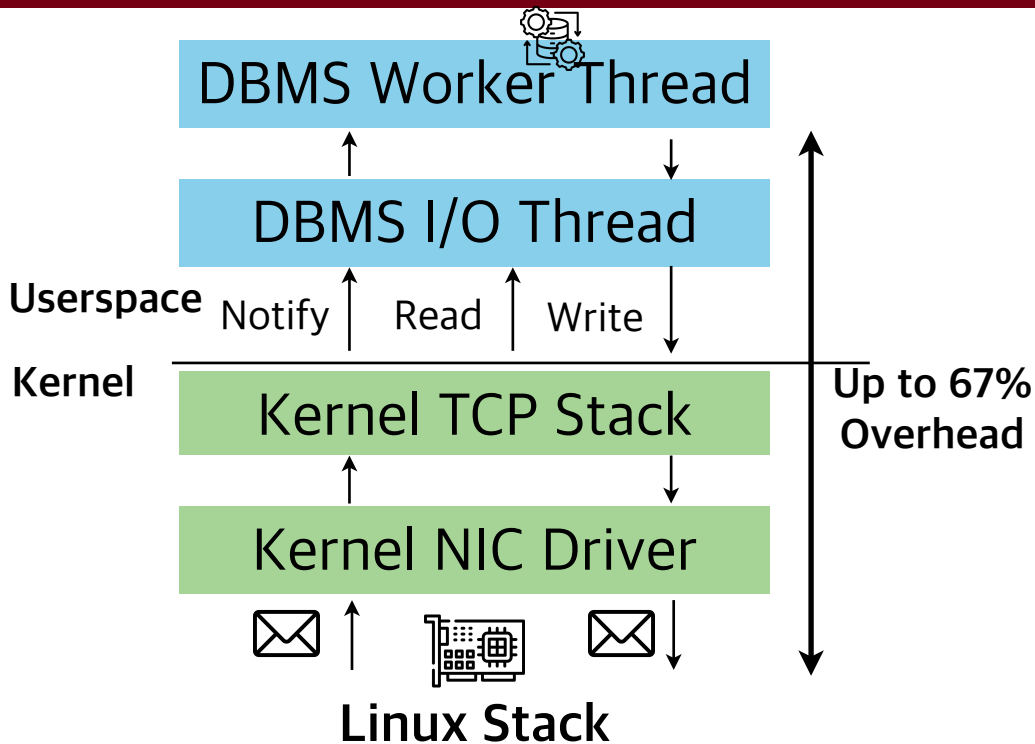
Kernel stack vs kernel bypass



Kernel stack vs kernel bypass

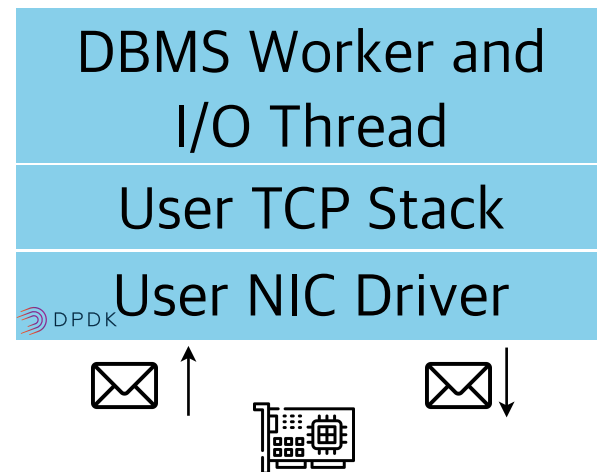


Kernel stack vs kernel bypass



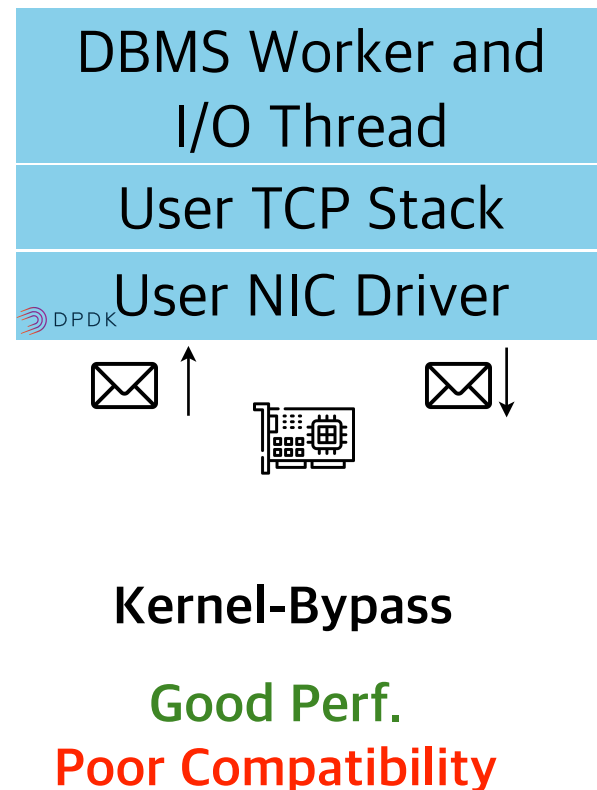
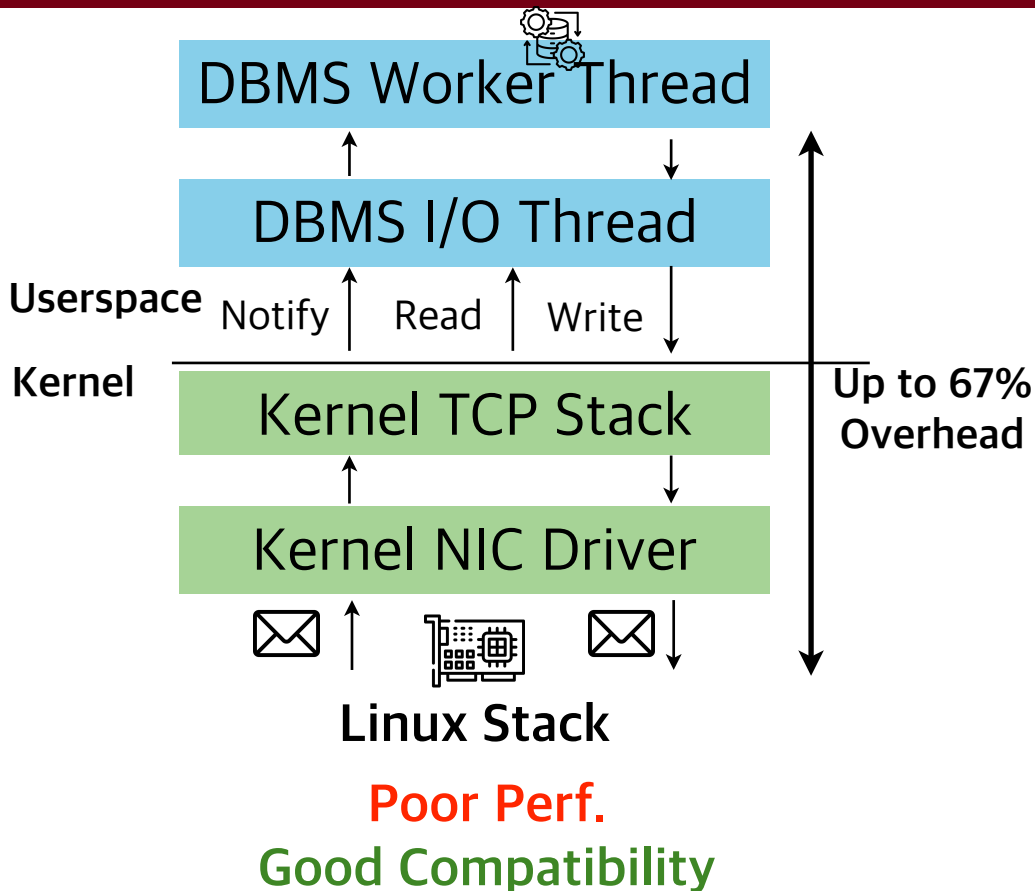
Poor Perf.

Good Compatibility

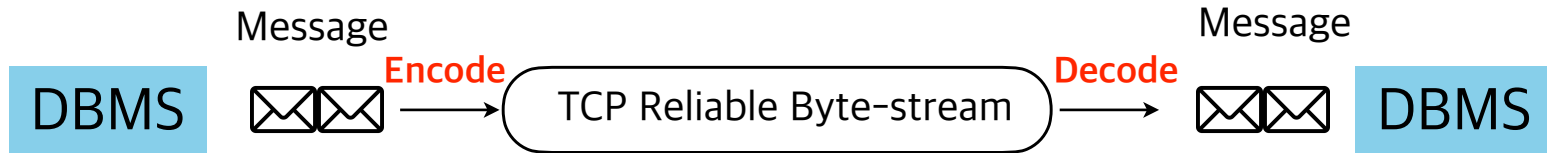


Kernel-Bypass

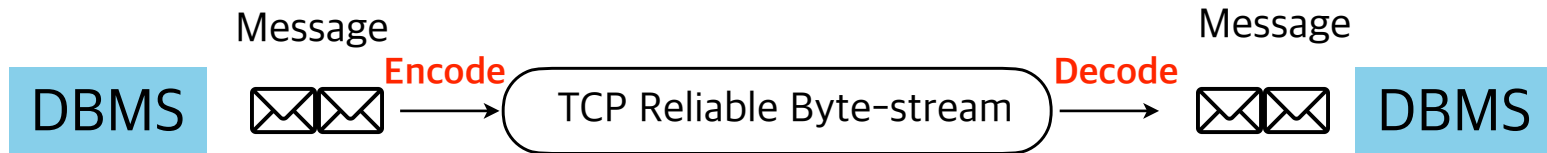
Kernel stack vs kernel bypass



Insight #1: Reliable messaging is often enough for DBMS



Insight #1: Reliable messaging is often enough for DBMS



Message-Oriented Communication

2PC

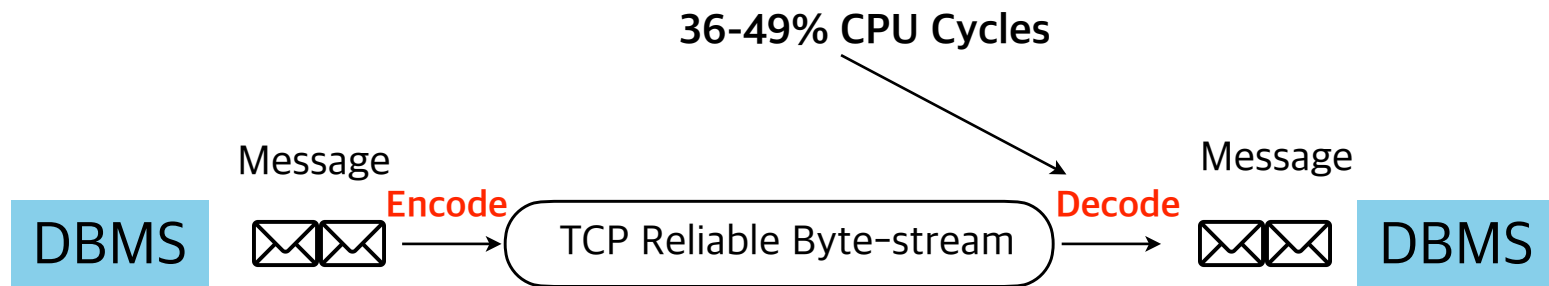
Paxos

2PL

GetPage@LSN

...

Insight #1: Reliable messaging is often enough for DBMS



Message-Oriented Communication

2PC

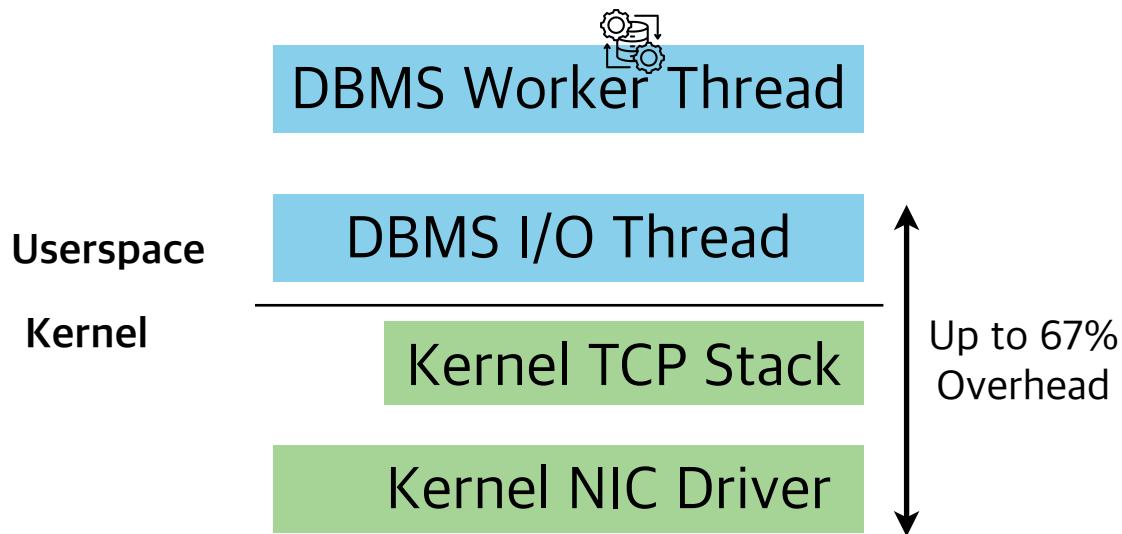
Paxos

2PL

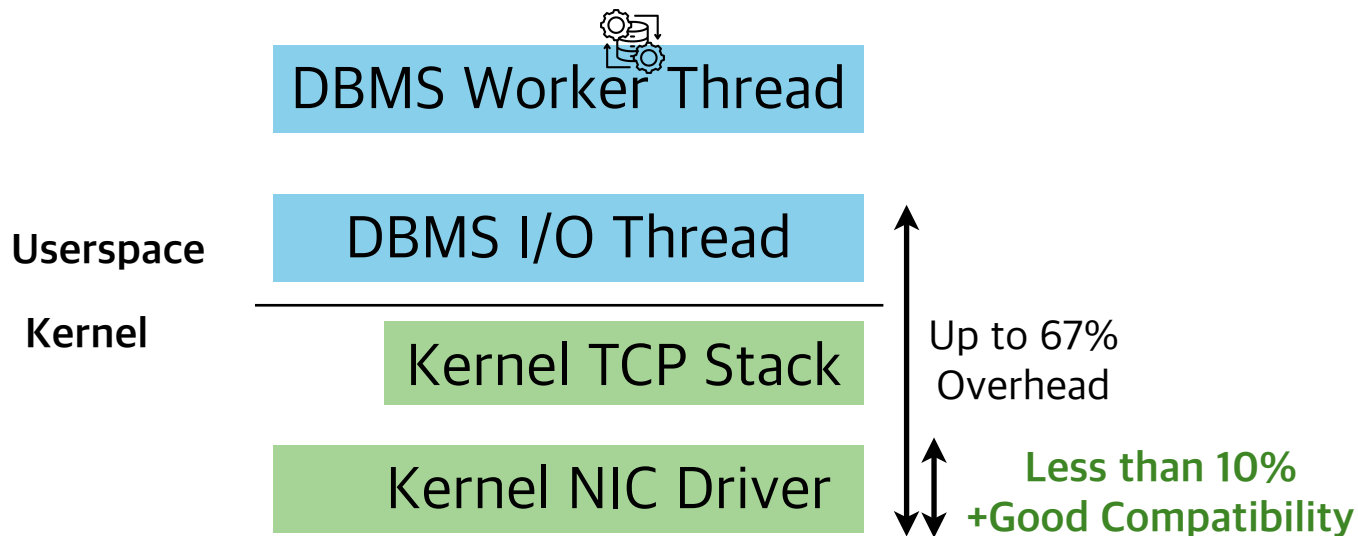
GetPage@LSN

...

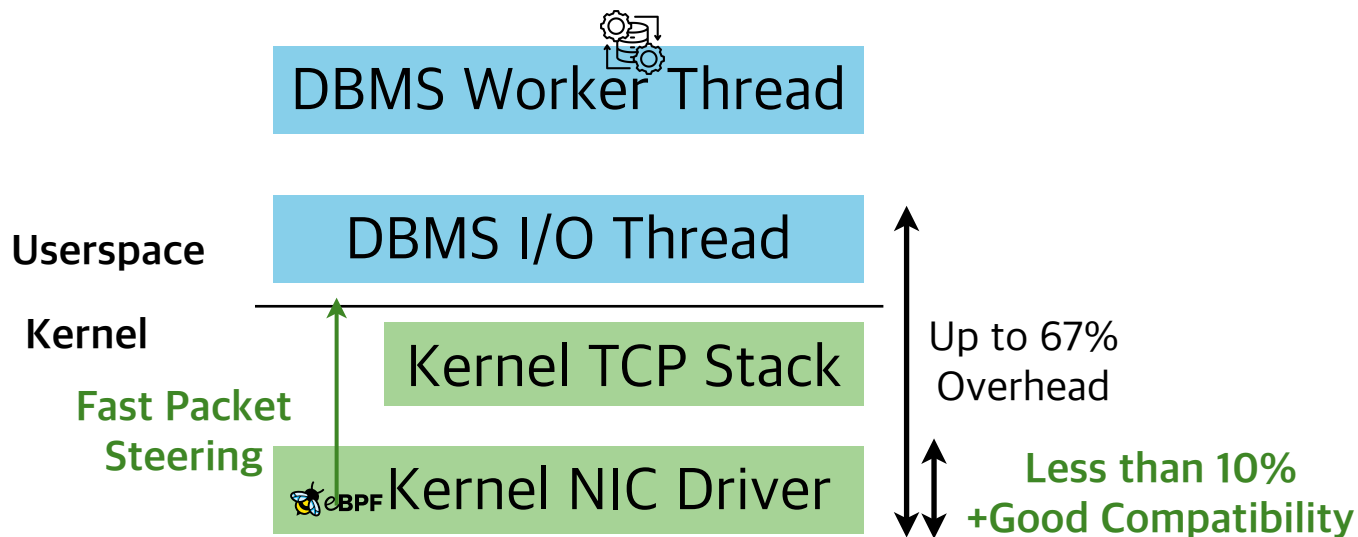
Insight #2: Kernel NIC Driver offers good compatibility with acceptable overhead



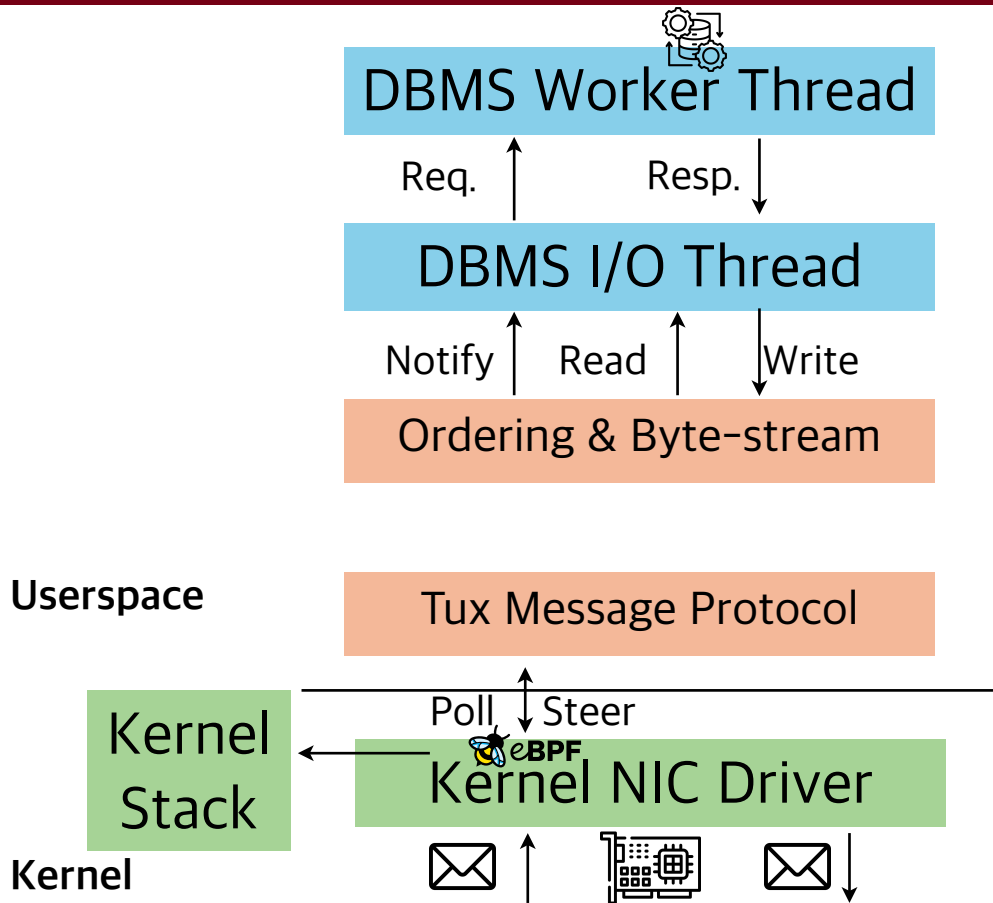
Insight #2: Kernel NIC Driver offers good compatibility with acceptable overhead



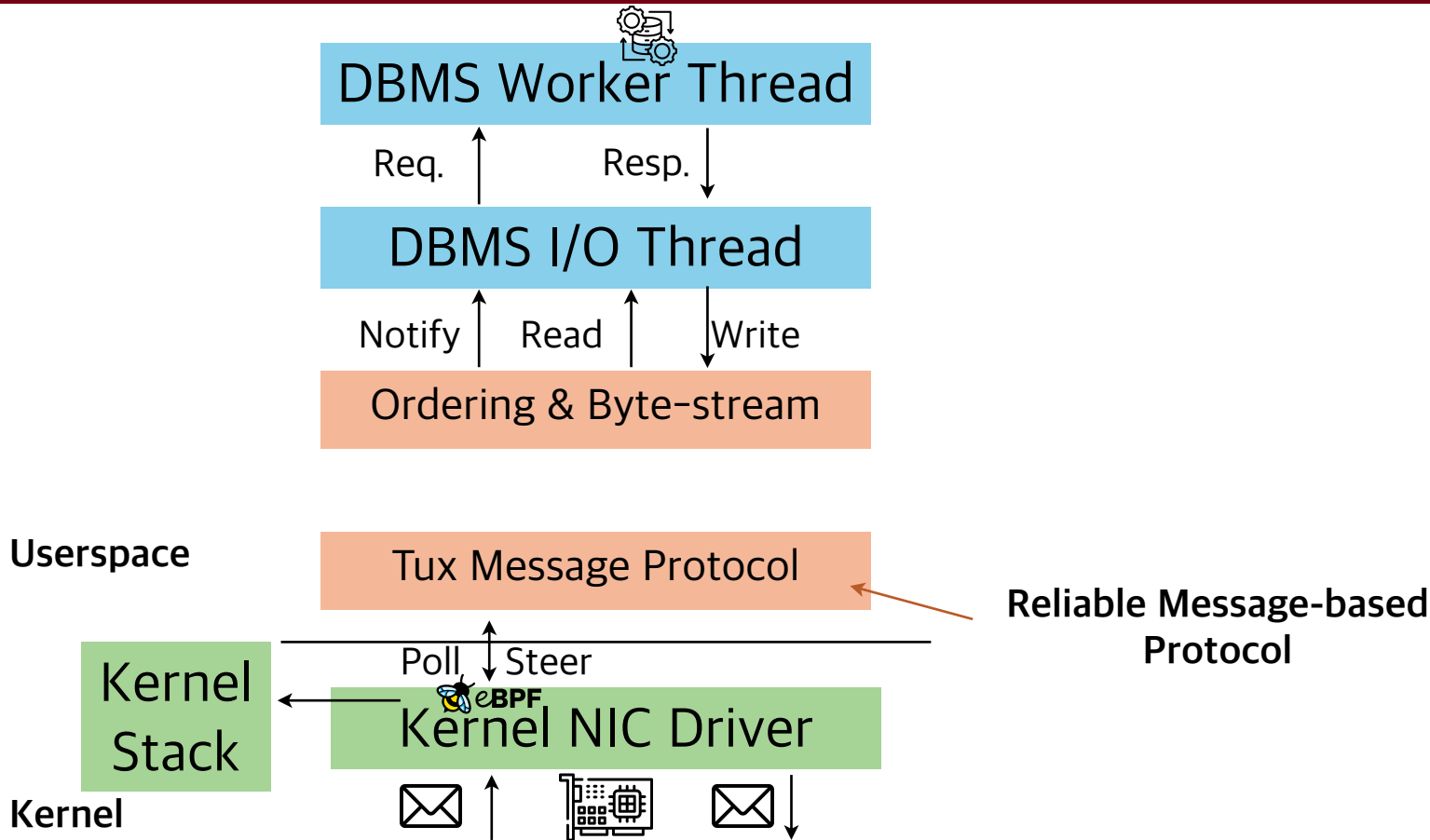
Insight #2: Kernel NIC Driver offers good compatibility with acceptable overhead



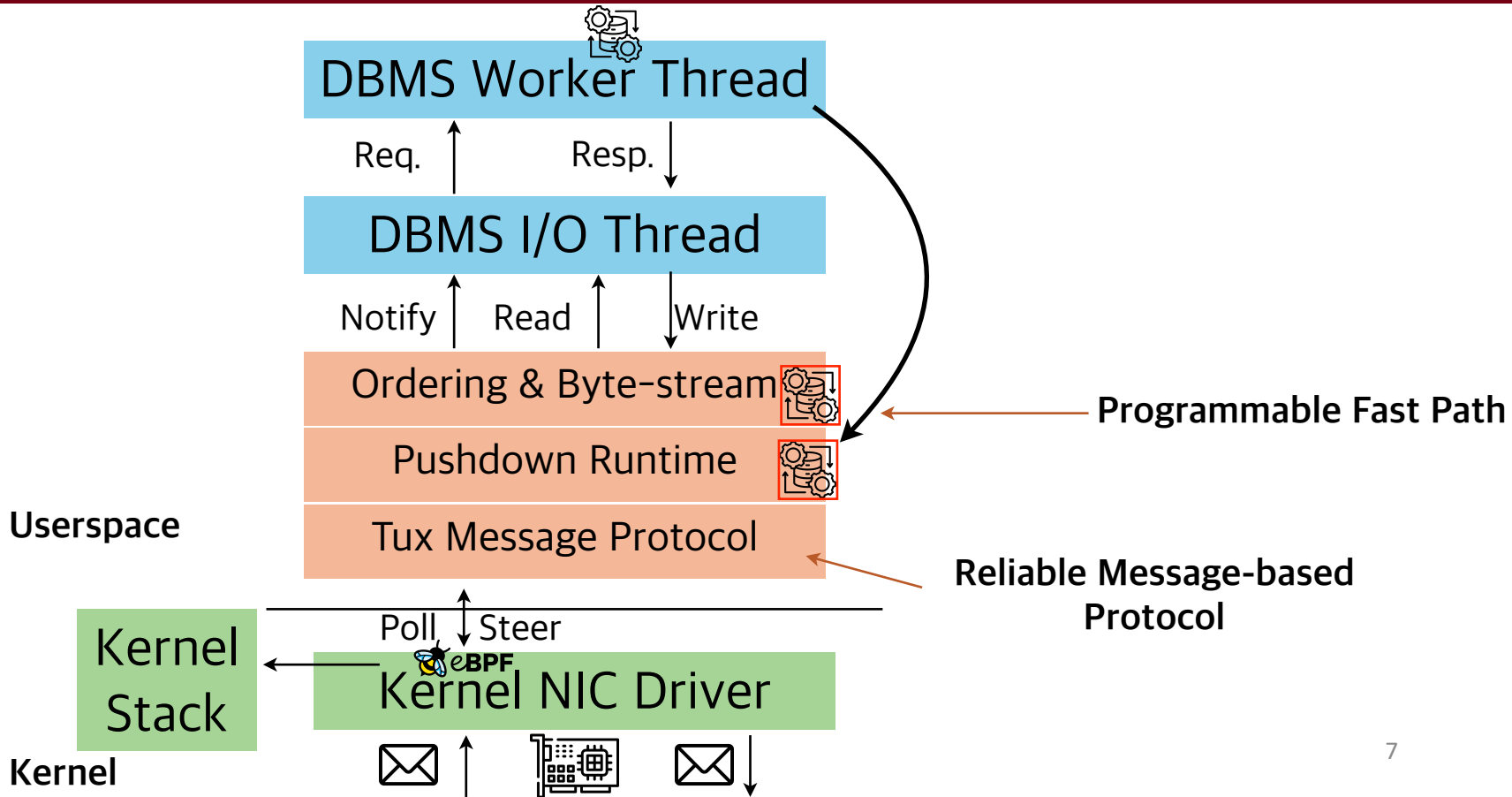
Tux: partial kernel-bypass programmable network stack



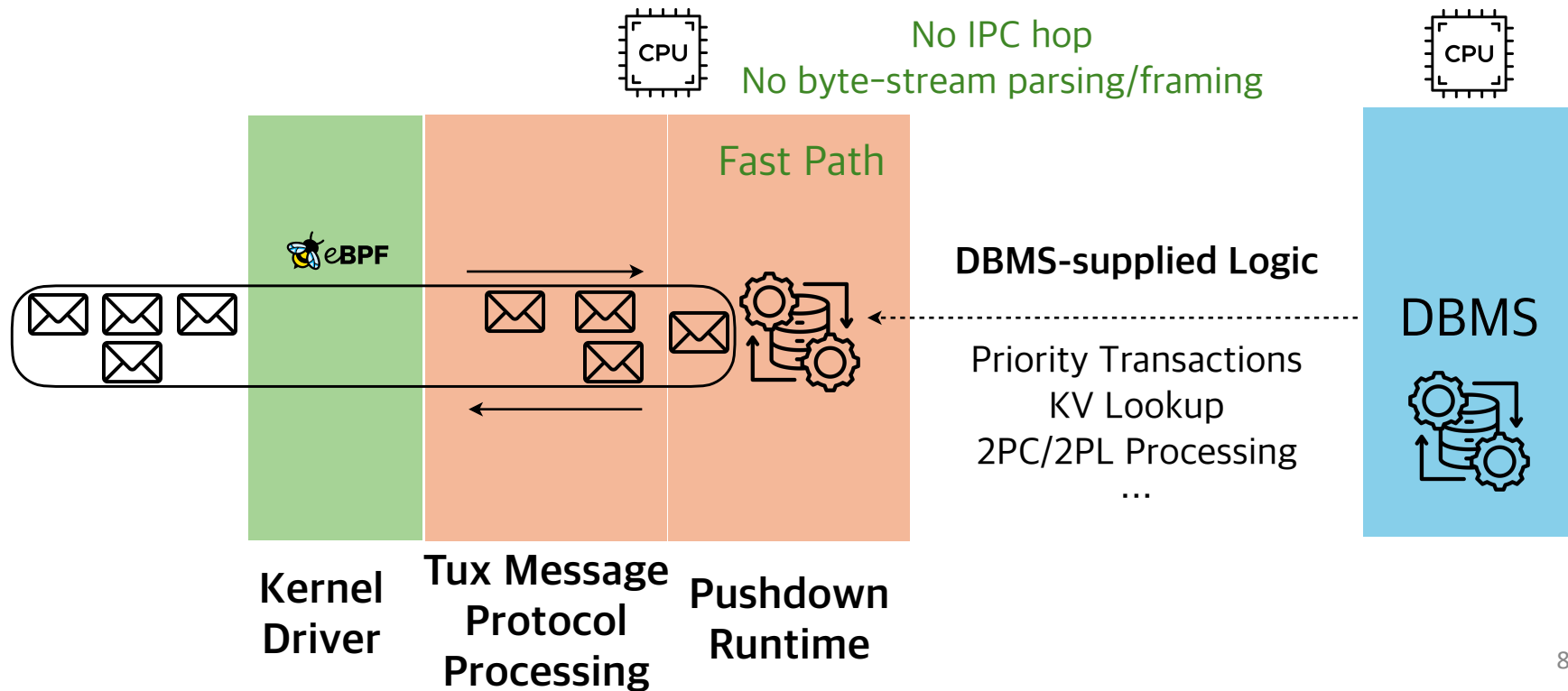
Tux: partial kernel-bypass programmable network stack



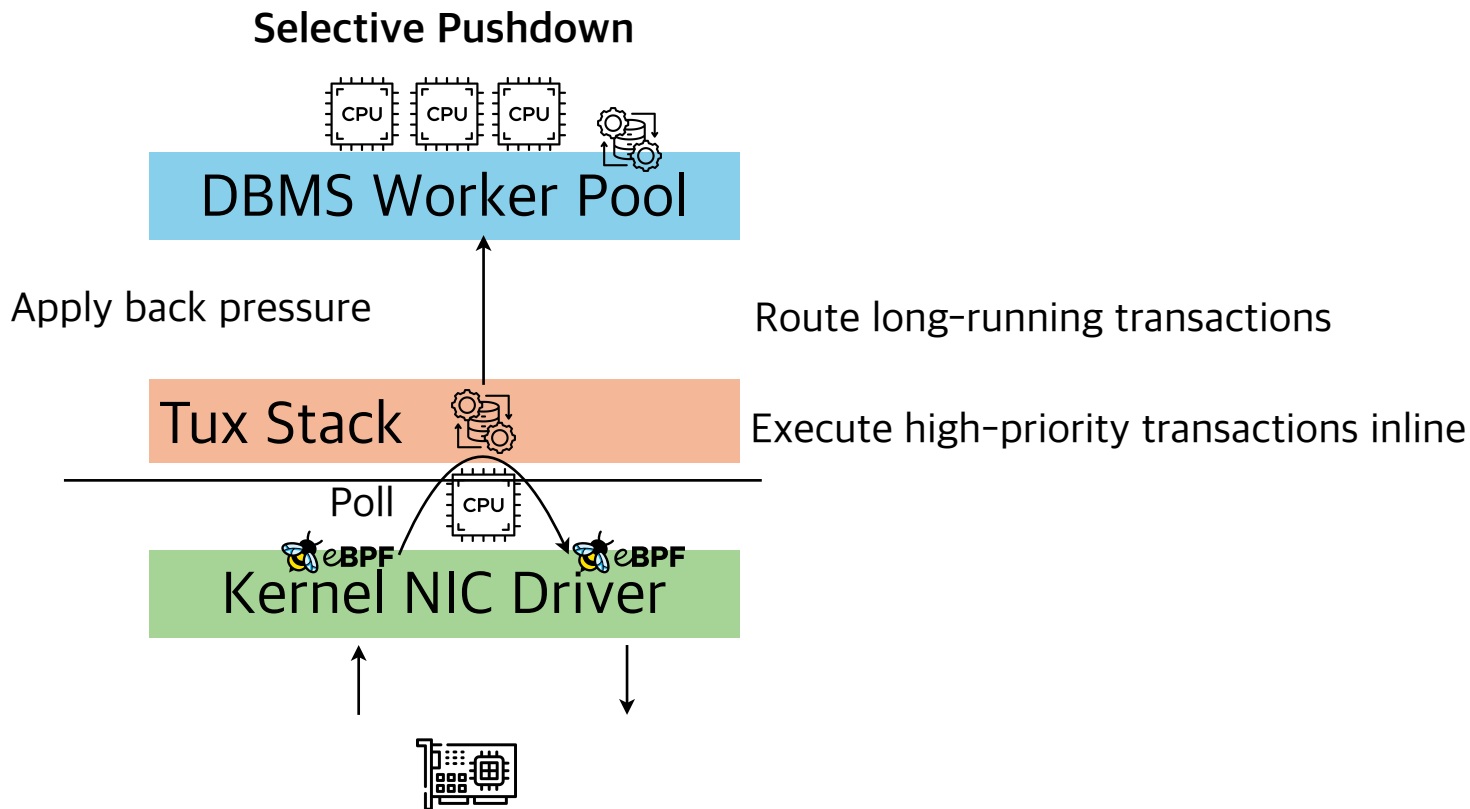
Tux: partial kernel-bypass programmable network stack



Pushdown executes DBMS logic in network stack



Programmability enables DB-assisted scheduling

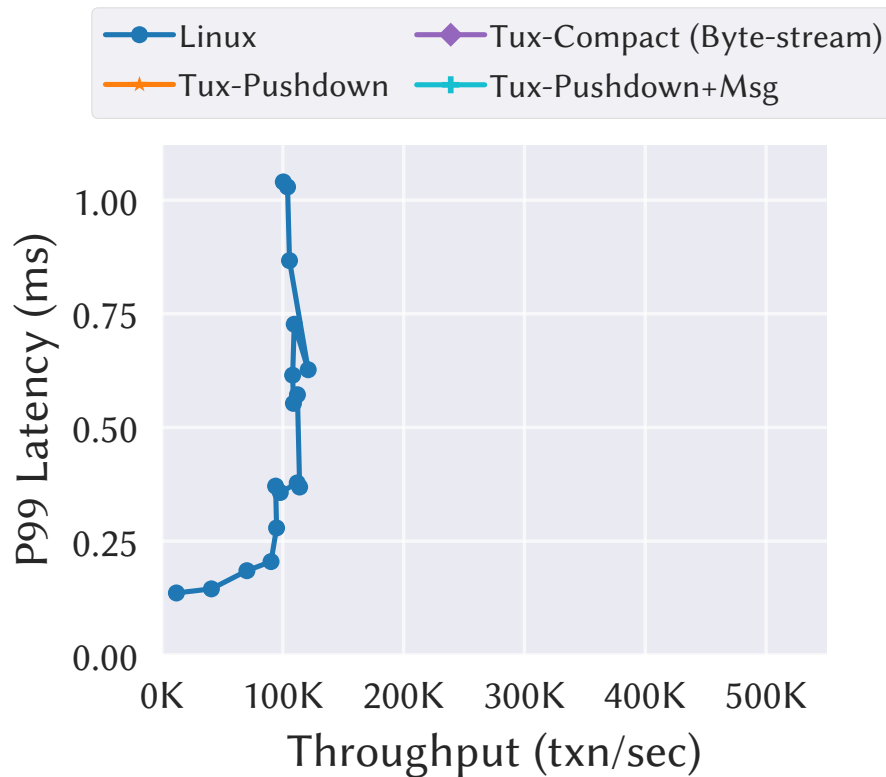


Challenges for Tux stack

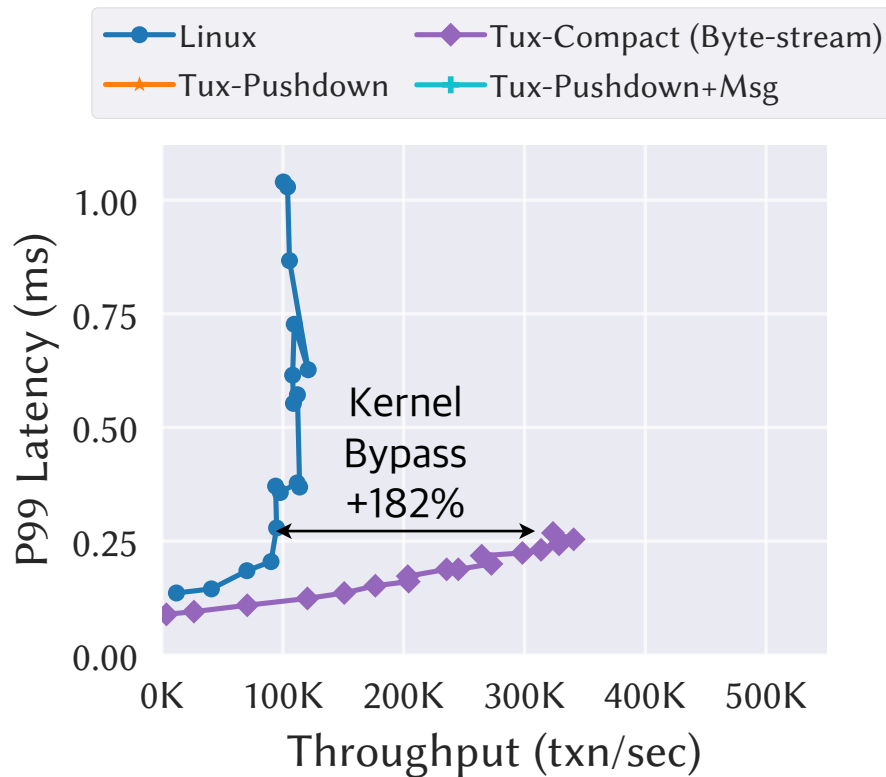
Challenge	Tux Solution
Kernel-crossing overhead	Read-side/Write-side batching
Starvation in pushdown logic	Preemptible co-routine
Storage I/O in pushdown logic	Async io_uring
Thread local storage support	Per-Routine TLS

Read Paper for More

Impact of message-based networking and pushdown

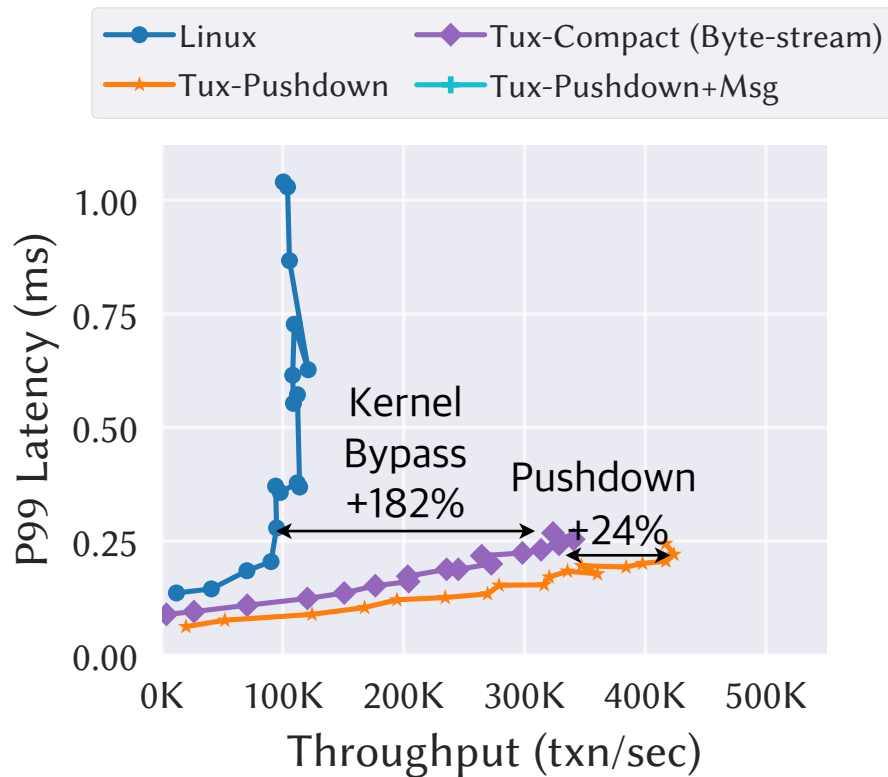


Impact of message-based networking and pushdown

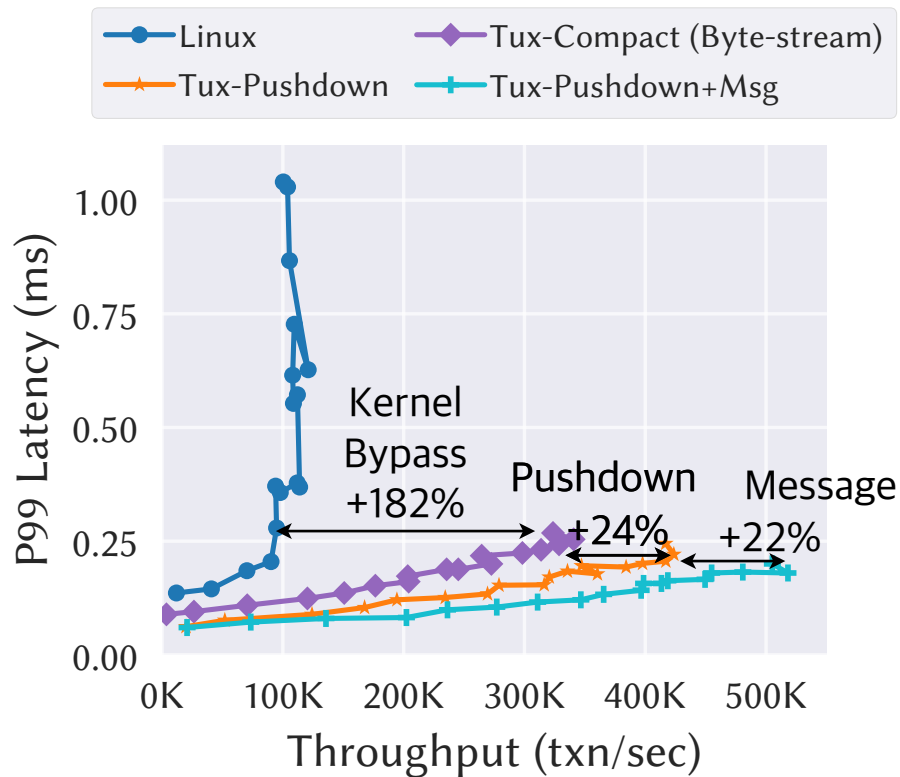


Networked LeanStore on YCSB workloads (AWS EC2 c5n.4xlarge)

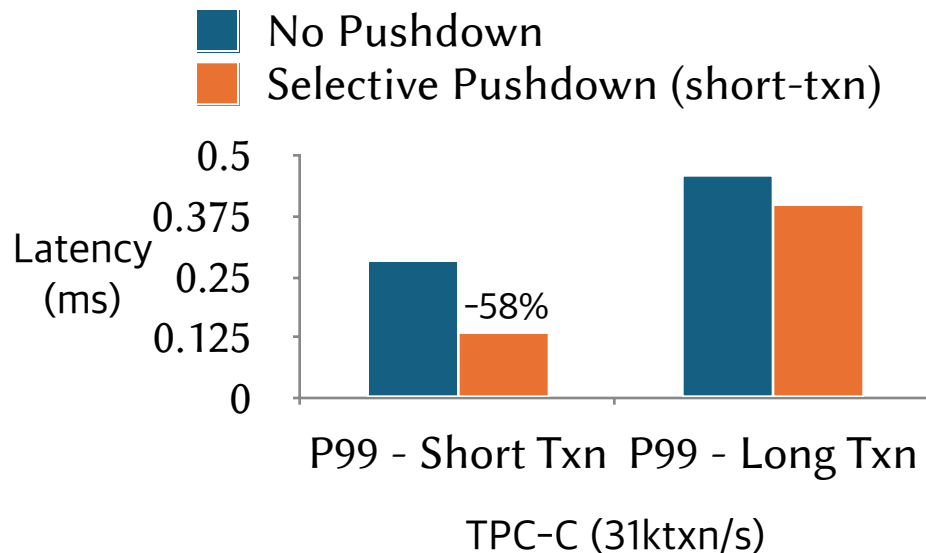
Impact of message-based networking and pushdown



Impact of message-based networking and pushdown



DB-Assisted scheduling helps priority transactions



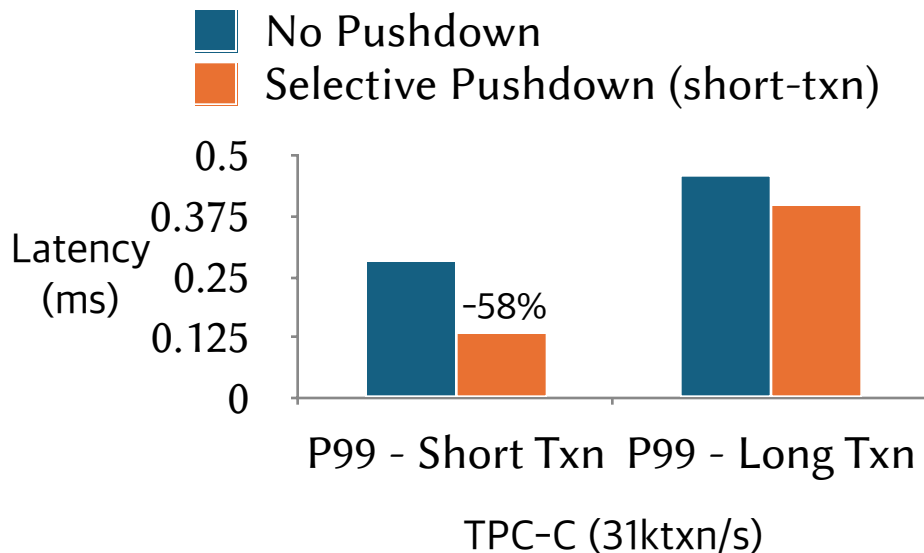
Short-running:

New-Order, Payment, Order-status

Long-running:

Stock-Level, Delivery

DB-Assisted scheduling helps priority transactions



Short-running:

New-Order, Payment, Order-status

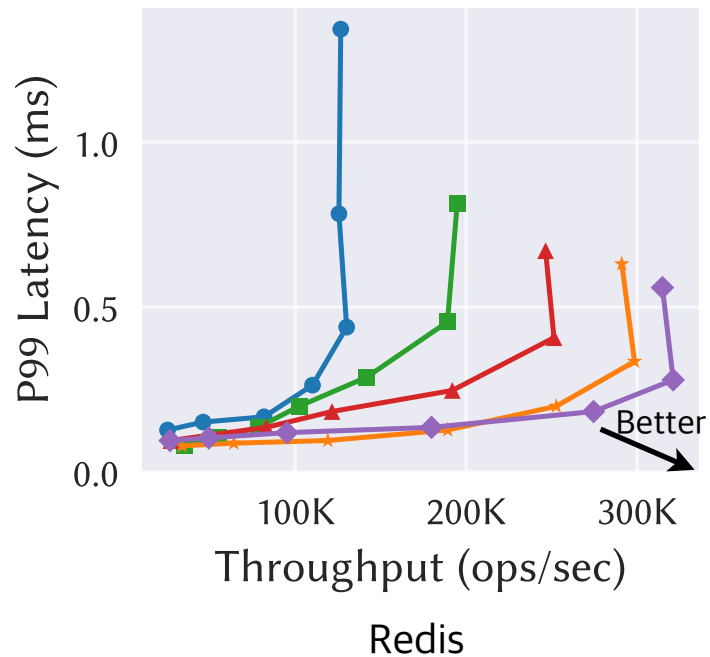
Long-running:

Stock-Level, Delivery

Selective Pushdown helps reduce high-priority transaction tail latency without degrading low-priority ones.

Tux matches/beats DPDK-based TCP stacks

Linux TAS+DPDK Tux-Compat
F-stack+DPDK Tux-Pushdown



Kernel Bypass Stack	Redis	VoltDB	Memcached
F-stack	678	1,300	-
Demikernel	1,958	-	-
TAS	0	-	0
Tux (Compatibility)	0	0	0
Tux (Pushdown)	63	-	59

Table 2: Code changes for porting DBMSes to kernel bypass stacks.

Similar trends on VoltDB,
ScyllaDB, Memcached,

Conclusion

- Tux makes DBMS-aware high-performance networking practical
 - **Programmable message-based fast path**: fast path for executing DBMS logic on reliable messages
 - **Compatible**: keeps kernel NIC drivers via recent high-performance eBPF traffic steering features

Code



Paper



BACKUP

Redis Latency Breakdown

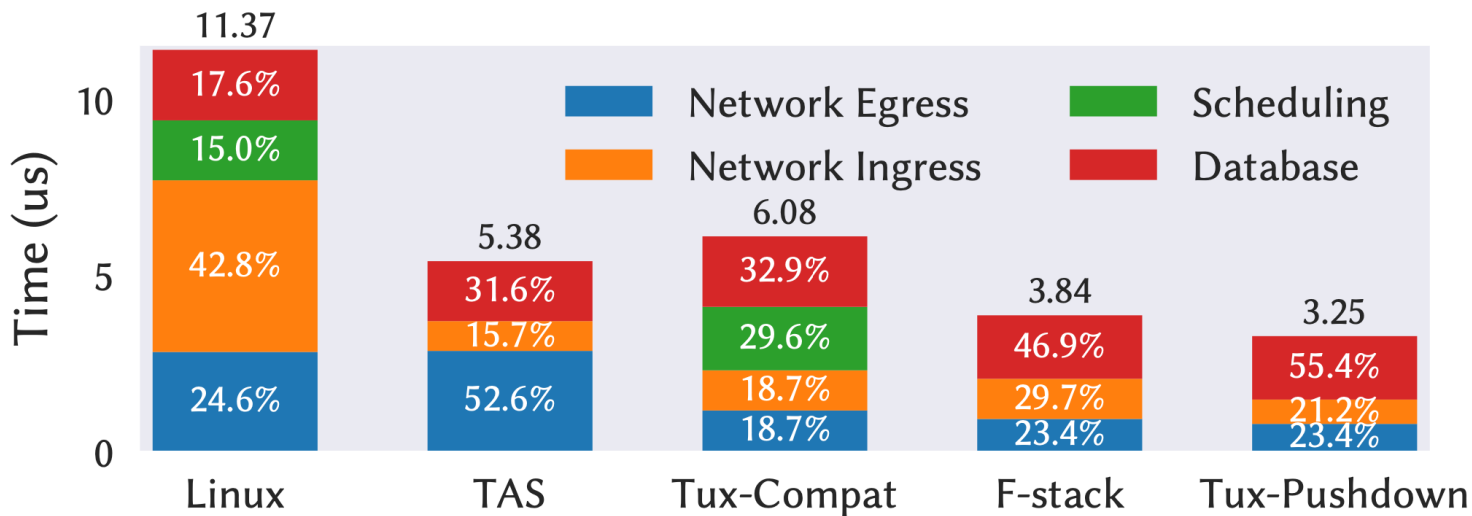


Figure 10: Server-side Latency Breakdown of Redis GET Request on Kernel Bypass Stacks. (Throughput = 50K ops/s on 1M 1KB-sized records, excluding polling, queueing, and off-CPU time).